PONTO 1 · VISÃO HOLÍSTICA

SDLC seguro quando a IA escreve o código

Os princípios da Janca, as técnicas da literatura
e os mecanismos do loop com IA, no mesmo ciclo.

O MOLDE

Três camadas, da regra à execução

Princípio	Que regra não quebra?	Janca, 2020
Técnica	Qual método nomeado acha o problema?	Literatura (quem, ano)
Mecanismo	O que executa a regra?	Playbook de SDLC com IA, 2026

A IA não muda a regra. Ela muda quem executa e em que velocidade.

O LIVRO INTEIRO, NUM QUADRO

Onze capítulos, cinco fases

FASE	CAPÍTULOS
Base	1 Fundamentos · 10 Aprendizado contínuo
Requisitos · Design	2 Requisitos · 3 Design seguro
Código	4 Código seguro · 5 Armadilhas comuns
Testes · Deploy	6 Testes e deploy · 8 Sistemas modernos
Manutenção · Programa	7 Programa de AppSec · 9 Bons hábitos

Parte I: o que saber para escrever código seguro. Parte II: o que fazer para ele continuar seguro.

PUSHING LEFT

Quanto mais tarde, mais caro



Bug nasce no código → **patch**

Falha nasce no desenho → **reprojeto**

O QUE A IA MUDA

Código ficou barato. Desenho errado, não.

O gargalo migra para **planejar, revisar e implantar**

A falha de design sai **mais rápido** e em mais lugares

Controle feito para código humano **deixa de bater**

Pushing Left fica **mais urgente**, não menos

FUNDAMENTOS · VALEM EM TODAS AS FASES

Doze fundamentos do capítulo 1

- CIA
- Assume breach
- Ameaça interna
- Defense in depth
- Least privilege
- Cadeia de suprimentos
- Sem obscuridade
- Menos superfície de ataque
- Sem hard coding
- Never trust, always verify
- Segurança usável
- Fatores de autenticação

O CICLO

Cinco fases, um loop, um artefato por fase

Requisitos	Design	Código	Testes	Deploy e manutenção
PLAN	DESIGN	BUILD	TEST	DEPLOY + MAINTAIN
`intent.md`	`spec.md`	`plan.md + diff`	`teste + eval`	`PR · incidente`

↺ o incidente vira uma nova intenção, e o ciclo recomeça

Fase: Janca · Estágio e artefato: playbook

RASTREABILIDADE

A mesma corrente, agora versionada

RACIOCÍNIO	Ativo	Classificação	Requisito	Controle	Teste
	↓	↓	↓	↓	↓
ARTEFATO	`intent.md`	`intent.md`	`spec.md`	`plan.md + diff`	`teste + PR`

Cada estágio commita o que o próximo lê. O histórico vira **trilha de auditoria**.

FASE 1 · REQUISITOS ↔ PLAN

O que o sistema deve garantir

PRINCÍPIOS

Verificabilidade

Rastreabilidade

Negação explícita

TÉCNICAS

Abuse case
McDermott e Fox, 1999

Misuse case
Sindre e Opdahl, 2000

Attack tree
Schneier, 1999

MECANISMO COM IA

Intenção conversada com o agente

Template de `intent.md` pede requisito negativo e critério de aceite

Saída: requisito com teste. Sem teste, não entrou no ciclo.

FASE 2 · DESIGN

Desenho seguro antes do código

PRINCÍPIOS

Least privilege

Defense in depth

Fail secure

Zero trust

TÉCNICAS

Modelagem de ameaças
Shostack, 2014

DFD
DeMarco, 1979

Superfície de ataque
Manadhata e Wing, 2004

MECANISMO COM IA

`spec.md` gerada sob skill de segurança

Conflito com política vai para o dono decidir

Saída: desenho com ameaças e mitigações. Sem mitigação, não saiu da fase.

FASE 3 · CÓDIGO ↔ BUILD

Sem repetir falha conhecida

PRINCÍPIOS

Never trust input

Nada hard coded

Identidade e sessão não se improvisam

TÉCNICAS

Inspeção formal
Fagan, 1976

Secure code review
OWASP Code Review Guide

SAST no commit
ver ponto 7

MECANISMO COM IA

Modo plano: lê sem editar → `plan.md`

Hook bloqueia segredo no diff e caminho protegido

Saída: código revisado, com teste que prova o requisito.

CAPÍTULO 4 NA PRÁTICA

Dois casos que valem para tudo

UPLOAD DE ARQUIVO · ASSUME O PIOR

Valida tipo e tamanho

Renomeia; o usuário não escolhe o caminho

Guarda longe da aplicação e escaneia

Allowlist: só os tipos menos perigosos

ERRO E LOG · FAIL-CLOSED

Nada de stack trace na tela

Desfaz a transação e fecha o que abriu

Loga para quem responde e para quem audita

No formato que o SIEM espera

O hook do agente segue a mesma regra: **na dúvida, nega**.

FASE 4 · TESTES ↔ TEST

Achado com evidência

PRINCÍPIOS

Achado sem prova não é achado

Bug bar decide a release

Testar os sete alvos, não só o fonte

TÉCNICAS

Fuzzing
Miller et al., 1990

SAST · DAST · SCA
ver ponto 7

Pentest

MECANISMO COM IA

Teste que falha primeiro, protegido contra edição

Todo incidente vira eval permanente no CI

Saída: achado com severidade, prova e decisão.

CAPÍTULO 6 · O QUE SE TESTA

Sete alvos, não só o código

- **Código**
- **Aplicação**
- **Infraestrutura**
- **Banco de dados**
- **APIs e web services**
- **Integrações**
- **Rede**
- **+ Deploy**

O agente prova o próprio trabalho no código. **Os outros seis alvos continuam com o time.**

FASE 5 · DEPLOY E MANUTENÇÃO ↔ DEPLOY + MAINTAIN

Detectar e responder em produção

PRINCÍPIOS

Assume breach

Log sem segredo

Alerta sem resposta não existe

TÉCNICAS

Resposta a incidentes
NIST SP 800-61

Gestão de vulnerabilidades
ver ponto 10

MECANISMO COM IA

Agente vai até o portão de produção; uma pessoa nomeada passa

1σ registra · 2σ diagnostica · 3σ propõe PR

Saída: incidente contido e lição virando `intent.md`.

O hábito da Janca e o mecanismo que o executa

HÁBITO (JANCA)	MECANISMO NO LOOP COM IA
Fire drill	Rollback é o caminho mais ensaiado
Varredura contínua	Scans de segurança agendados em todos os repositórios
Resposta a incidentes	Agente no canal do incidente; post-mortem vira `intent.md`
Inventário	Artefatos versionados: o que existe está no repositório

O AGENTE TAMBÉM É SUPERFÍCIE DE ATAQUE

Os princípios da Janca aplicados ao agente

PRINCÍPIO	GUARDRAIL NO AGENTE
Least privilege	Permissão mínima, token de CI curto e escopado
Sem segredo em código	Leitura de `.env` e credenciais negada
Zero trust + fail secure	Sandbox com allowlist de domínio que falha fechado
Ameaça interna	Quem escreveu o código não aprova; sem rota direta para a main
Cadeia de suprimentos	Só hooks, plugins e MCP gerenciados

SEGURANÇA USÁVEL

Aconselhar não é impedir

CONTROLE	MECANISMO	EFEITO
Advisory	Skill, `CLAUDE.md`	Conformidade **provável**
Enforced	Hook, permissão, branch protection	Violação **quase impossível**

Controle que atrapalha é contornado (cap. 1).
A Janca chama de **Security as Code** (cap. 8): a regra vive no sistema, não no hábito.

REGRAS DE OURO

Uma de cada lado, mesma ideia

JANCA	LOOP COM IA
Bug se corrige com patch; falha, com reprojeto	Plano aprovado antes de qualquer edição
Design review por quem não desenhou	O agente que escreveu não aprova
Plano de resposta não ensaiado falha	Rollback é o caminho mais ensaiado
Lição vira requisito do próximo ciclo	Incidente vira `intent.md` e eval

O loop continua rodando.
O julgamento humano fica acima dele.

Janca, T. (2020). *Alice and Bob Learn Application Security*. Wiley.

Claxton, L. (2026). *The AI-native SDLC playbook*. Anthropic.

Shostack, A. (2014). *Threat Modeling*. Wiley.

McDermott, J.; Fox, C. (1999). Abuse case models.

Sindre, G.; Opdahl, A. (2000). Misuse cases.

Schneier, B. (1999). Attack trees.

DeMarco, T. (1979). *Structured Analysis*.

Manadhata, P.; Wing, J. (2004). Attack surface.

Fagan, M. (1976). Design and code inspections.

Miller, B. et al. (1990). Fuzz testing.

NIST SP 800-61. Incident handling guide.